LECTURE 9

Transformers and Attention: The Technical Treatment

Reference Text

Embeddings, sinusoidal positional encoding, scaled dot-product and multi-head attention, and every block of Vaswani et al. (2017) — worked by hand on "enemy tank advances"

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 How to Use This Text **3**

2 Why Transformers **3**

2.1 What held recurrent networks back . . . 3

2.2 Every word sees every word . . . 3

3 Tokens and Embeddings **4**

3.1 Tokenisation . . . 4

3.2 The embedding matrix . . . 4

3.3 Directions carry meaning: a worked analogy . . . 4

3.4 Word space and context space . . . 5

4 Positional Encoding **5**

4.1 Attention is order-blind . . . 5

4.2 The sinusoidal formula . . . 5

4.3 Worked example: $d_{\text{model}} = 4$ . . . 6

4.4 Why sinusoids: relative position is a rotation . . . 6

4.5 Position changes what attention sees . . . 7

5 Queries, Keys and Values **7**

6 Scaled Dot-Product Attention **7**

6.1 Worked example: set-up . . . 8

6.2 Step 1 — queries, keys and values . . . 8

6.3 Step 2 — scores and scaling . . . 8

6.4 Step 3 — softmax . . . 8

6.5 Step 4 — blend the values . . . 8

6.6 Why divide by $\sqrt{d_k}$ . . . 9

6.7 Masked (causal) self-attention . . . 9

7 Multi-Head Attention **9**

7.1 Worked example: a second head . . . 9

7.2 Concatenate and project . . . 10

7.3 Parameter count of one attention layer . . . 10

7.4 What heads learn . . . 10

8 The Architecture of *Attention Is All You Need* **10**

8.1 Following one sentence through the figure . . . 11

8.2 **(1) (2)** Embedding and positional encoding . . . 11

8.3 **(3)** The encoder layer . . . 11

8.4 **(4)** Decoder input and masked self-attention . . . 12

8.5 **(5)** Encoder–decoder (cross) attention 12
8.6 **(6)** Linear and softmax 12
8.7 Three uses of the same attention 13

9 The Paper's Settings and Results **13**

10 Three Families of Transformer Models **14**

11 Generation and Training **14**
11.1 Next-token generation 14
11.2 Training 14

12 Defence and Wargaming Applications **15**

13 Common Misconceptions **15**

14 Summary of Equations **16**

15 Check Your Understanding **16**

1 How to Use This Text

This text accompanies the Lecture 9 slides and gives the full mathematics behind the intuitions of Lecture 8. It assumes matrix multiplication, the softmax function, and backpropagation from Lecture 6. One running example is carried through every stage: the three-token sentence "*enemy tank advances*" with four-dimensional toy vectors and two-dimensional attention heads, so that every number can be checked with a calculator. The final sections follow the same sentence through each block of Figure 1 of *Attention Is All You Need* (Vaswani et al., 2017). The lecture is the densest in the course; read Sections 2–6 after Part 1 of the session and the rest after Part 2.

2 Why Transformers

2.1 What held recurrent networks back

A recurrent neural network (RNN) reads one token per step and keeps a hidden state,

$$h_t = \tanh(W_h h_{t-1} + W_x x_t).$$

Three consequences follow. **Sequential computation:** h_2 cannot be computed until h_1 exists, so a 1,000-word report needs 1,000 dependent steps per layer and training cannot be parallelised across the sentence. **Forgetting:** information from early words must survive many multiplications by W_h; gradients shrink or explode along the way (Lecture 6's vanishing-gradient problem). **Bottleneck:** the entire past is squeezed into one vector h_t. From "enemy" to "advances" is two hops; for words 500 apart it is 500 hops.

2.2 Every word sees every word

The transformer drops recurrence and relies on attention alone. Stack the token vectors as the rows of a matrix X ($n \times d_{\text{model}}$); then one matrix product computes all queries at once,

$$\underbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}}_{X} \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}}_{W^Q} = \underbrace{\begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}}_{Q},$$

and $QK^\top$ scores all $3 \times 3 = 9$ word pairs in one more. On a GPU the rows are computed simultaneously, and every word is *one* hop from every other.

Layer type	Complexity per layer	Sequential operations	Maximum path length
Self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolutional	$O(knd^2)$	$O(1)$	$O(\log_k n)$

Table 1 of Vaswani et al. (2017); n sequence length, d width, k kernel size.

Sequential operations count steps that must wait for one another; $O(1)$ is the training-speed breakthrough. **Maximum path length** counts hops between the two most distant words; short paths make long-range dependencies easy to learn. The price is n^2: every word scores every word. For a sentence of $n \approx 30$ tokens with $d = 512$, $n^2 d \approx 0.46$ million operations against $nd^2 \approx 7.9$ million for a recurrent layer, so attention is cheaper; for a 10,000-token document $n^2 d \approx 51$ billion and attention becomes the dominant cost.

Intuition

the analyst Reading "The drone tracked the convoy because *it* was moving", an analyst does not reread the report word by word; they glance back to the few words that could resolve "it" and settle on "convoy". Change "moving" to "airborne" and they settle on "drone". Attention is this learned, soft cross-referencing: every word receives a weighted summary of the words most relevant to it, with weights that sum to 1.

3 Tokens and Embeddings

3.1 Tokenisation

Text is split into sub-word **tokens** from a fixed vocabulary. The original paper used **byte-pair encoding** (BPE) with about 37,000 tokens shared by English and German. BPE starts from single characters and repeatedly merges the most frequent adjacent pair into a new symbol until the vocabulary reaches the chosen size; common words end as one token and rare words as several ("Unidentified" → "Un" + "identified"). A rule of thumb for English is one token per three-quarters of a word. The model never sees letters, only integer IDs. Military jargon, call-signs and grid references are often split into many tokens, a hidden cost in context length and accuracy.

3.2 The embedding matrix

Definition — embedding layer

A learned matrix $E \in \mathbb{R}^{V \times d_{\text{model}}}$ with one row per vocabulary token. Embedding token i means selecting row i: $x = E_{i,:}$. Equivalently $x = \mathbf{e}_i^{\mathsf{T}} E$ for the one-hot vector $\mathbf{e}_i$.

$d_{\text{model}} = 512$ in the original transformer, 768 in BERT-base and 12,288 in GPT-3. Positions in the space are not designed by anyone: they emerge in training because words used in similar contexts must produce similar predictions. Similarity is measured by the dot product or by the cosine $\cos\theta = x \cdot y / (\|x\|\|y\|)$ — the same operation attention uses.

3.3 Directions carry meaning: a worked analogy

In trained embeddings, consistent relationships appear as consistent *directions*. To make the arithmetic visible, use four labelled toy dimensions (gender, human, feline, royalty); real dimensions have no labels.

word	gender	human	feline	royal
man	0.90	0.95	0.05	0.05
woman	−0.90	0.95	0.05	0.05
tiger	0.80	0.05	0.95	0.00
tigress	−0.75	0.10	0.90	0.05
lioness	−0.80	0.05	0.90	0.30
king	0.85	0.90	0.00	0.95
queen	−0.85	0.90	0.00	0.95
male	1.00	0.10	0.10	0.00
female	−1.00	0.10	0.10	0.00

Worked example — tiger − man + woman

The gender direction is $\overrightarrow{\text{woman}} - \overrightarrow{\text{man}} = (-1.80, 0, 0, 0)$. Adding it to tiger: $(0.80, 0.05, 0.95, 0) + (-1.80, 0, 0, 0) = (-1.00,\ 0.05,\ 0.95,\ 0.00)$.
Cosine similarity with the candidates (the three input words excluded): **tigress 0.991**, lioness 0.967, female 0.789, queen 0.416. The nearest word is *tigress*.
In the same space, king − man + woman → queen (0.999), and man − male + female → woman (0.995).

Pitfall — analogies in real embeddings

Real analogies hold only approximately, the input words must be excluded from the search (otherwise the answer is often an input word), and the same geometry absorbs biases present in the training text, such as associating occupations with one gender (Mikolov et al. 2013; Bolukbasi et al. 2016).

3.4 Word space and context space

The embedding table gives *one* vector per token: "tank" is the same in every sentence. After attention layers each token's vector has absorbed its neighbours, giving a *contextual embedding*. With "fuel" and "leaked" nearby, "tank" drifts towards *barrel, drum, container*; with "enemy" and "advanced", towards *APC, howitzer, armour*. This is why an intelligence summariser can tell an armoured tank from a fuel tank, or "fire" the weapon from "fire" the blaze.

4 Positional Encoding

4.1 Attention is order-blind

Attention treats its input as a *set*. Permuting the rows of X permutes the rows of Q, K, V and of the output in the same way, and changes nothing else: "Blue engages Red" and "Red engages Blue" would give the same vectors in a different order. Order must be injected. The paper adds a position vector to each embedding:

$$h_0 = \sqrt{d_{\text{model}}}\, E_{\text{tokens}} + PE.$$

Nothing is concatenated: position and meaning share the same d_{model} numbers. A good position code should be **unique** for each position, **bounded** (so it does not swamp meaning), give the **same relation** between positions 3 and 5 as between 103 and 105, and work for sequences **longer** than any seen in training. Using $p_t = t$ fails the second requirement; using t/n fails the third, because it depends on the length n.

4.2 The sinusoidal formula

Definition — sinusoidal positional encoding (Vaswani et al., Section 3.5)

$$PE_{(pos,\,2i)} = \sin\!\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\!\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

$pos = 0, 1, 2, \ldots$ is the position; i indexes a dimension *pair*, the even member taking the sine and the odd member the cosine of the same angle.

The angular frequency of pair i is $\omega_i = 10000^{-2i/d_{\text{model}}}$; it falls geometrically from 1 to almost $1/10000$, so the wavelengths $2\pi/\omega_i$ grow from $2\pi \approx 6.3$ tokens to about 62,800.

pair i	ω_i	wavelength (tokens)
0	1	6.3
64	0.1	63
128	0.01	628
255	≈ 0.0001	$\approx$ 60,600

$d_{\text{model}} = 512$.

Intuition

a clock The fast pairs are a second hand: they change a lot between neighbouring positions and tell neighbours apart. The slow pairs are an hour hand: they barely move between neighbours but distinguish the start of a long report from its end. Read together, the hands give a unique time — a unique position.

4.3 Worked example: $d_{\text{model}} = 4$

With four dimensions there are two pairs: $\omega_0 = 1$ and $\omega_1 = 10000^{-2/4} = 0.01$.

	pos	$\sin pos$	$\cos pos$	$\sin 0.01pos$	$\cos 0.01pos$
enemy	0	0.0000	1.0000	0.0000	1.0000
tank	1	0.8415	0.5403	0.0100	1.0000
advances	2	0.9093	−0.4161	0.0200	0.9998

The fast pair moves $0 \to 0.84 \to 0.91$; the slow pair $0 \to 0.01 \to 0.02$. Adding to the toy embeddings E (the $\sqrt{d_{\text{model}}} = 2$ scaling omitted for simplicity):

$$\underbrace{\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix}}_{E} + \underbrace{\begin{pmatrix} 0 & 1 & 0 & 1 \\ 0.84 & 0.54 & 0.01 & 1 \\ 0.91 & -0.42 & 0.02 & 1 \end{pmatrix}}_{P} = \underbrace{\begin{pmatrix} 1.00 & 1.00 & 1.00 & 1.00 \\ 0.84 & 1.54 & 0.01 & 2.00 \\ 1.91 & -0.42 & 0.02 & 2.00 \end{pmatrix}}_{E+P}.$$

Each row now carries *what* the word is and *where* it is.

4.4 Why sinusoids: relative position is a rotation

Derivation — a shift by k positions is a linear map

For one pair with frequency ω, the angle-addition formulas give

$$\sin \omega(pos + k) = \sin \omega pos \cos \omega k + \cos \omega pos \sin \omega k,$$
$$\cos \omega(pos + k) = \cos \omega pos \cos \omega k - \sin \omega pos \sin \omega k,$$

that is,

$$\begin{pmatrix} \sin \omega(pos+k) \\ \cos \omega(pos+k) \end{pmatrix} = \begin{pmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{pmatrix} \begin{pmatrix} \sin \omega pos \\ \cos \omega pos \end{pmatrix}.$$

The matrix depends only on the offset k, so PE_{pos+k} is a fixed linear function of PE_{pos} for every pos. The paper's reason for the choice: the model "could easily learn to attend by relative positions".

Check ($\omega = 1$, $pos = 3$, $k = 2$): rotating $(\sin 3, \cos 3) = (0.1411, -0.9900)$ by angle 2 gives $(0.1411 \cos 2 + (-0.9900) \sin 2,\ -0.9900 \cos 2 - 0.1411 \sin 2) = (-0.9589,\ 0.2837) = (\sin 5, \cos 5)$.

A second consequence: $PE_{pos} \cdot PE_{pos+k} = \sum_i [\sin \omega_i pos \sin \omega_i(pos+k) + \cos \omega_i pos \cos \omega_i(pos+k)] = \sum_i \cos(\omega_i k)$, which depends *only* on k. With $d_{\text{model}} = 64$ the dot product is 32 at $k = 0$, 30.92 at $k = 1$, and 23.50 at $k = 5$ — the same for positions 5 and 10 as for 30 and 35. Nearby positions look more alike than distant ones. Every value of the encoding lies in $[-1, 1]$, and every row (position) is different: a unique barcode. The paper also tried learned position vectors with nearly identical results (its Table 3, row E) and kept sinusoids because they may extrapolate to longer sequences; BERT and GPT-2 learn theirs.

4.5 Position changes what attention sees

Using the attention weights of the worked example below, compare "enemy tank advances" with the swapped "tank enemy advances". **Without** positional encoding the swapped sentence produces exactly the same weights, merely re-ordered: enemy→tank is 0.77 and tank→enemy 0.58 in both orders. **With** the sinusoidal codes added, the weights change: enemy→tank is 0.72 in the original order and 0.96 when swapped; tank→enemy rises from 0.41 to 0.78. Word order now affects meaning.

5 Queries, Keys and Values

Intuition

an intelligence archive A **query** is the request slip: "what am I looking for?". Every file carries a **key**, its index label: "what do I contain?". Its **value** is the content handed over. A database returns the single exact match; attention is a *soft* lookup that returns a blend of all values, weighted by how well each key matches the query. Because the blend is differentiable it can be trained by backpropagation.

Each token's vector is projected three ways by learned matrices:

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V,$$

with shapes X: $n \times d_{\text{model}}$; W^Q, W^K: $d_{\text{model}} \times d_k$; W^V: $d_{\text{model}} \times d_v$; Q, K: $n \times d_k$; V: $n \times d_v$ (the paper uses $d_k = d_v = 64$). One word plays three roles: "tank" as a query might seek "who owns me?"; as a key it advertises "I am an armoured vehicle"; as a value it carries its meaning to whoever attends to it. W^Q, W^K, W^V are ordinary weights, learned by backpropagation like those of Lectures 6 and 7. In **self**-attention Q, K and V all come from the same sentence.

6 Scaled Dot-Product Attention

Definition — scaled dot-product attention (Vaswani et al., Figure 2, left)

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V,$$

where $\text{softmax}(s)_j = e^{s_j} / \sum_m e^{s_m}$ is applied row by row. Read the paper's figure bottom to top: MatMul ($QK^\top$, an $n \times n$ score table), Scale ($1/\sqrt{d_k}$), optional Mask (decoder only), SoftMax (each row to weights summing to 1), MatMul with V (blend the values).

6.1 Worked example: set-up

Three tokens, $d_{\text{model}} = 4$, $d_k = d_v = 2$. The integer toy vectors may be thought of as embedding plus position already combined:

$$X = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \begin{matrix} \leftarrow \text{enemy} \\ \leftarrow \text{tank} \\ \leftarrow \text{advances} \end{matrix} \quad W^Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{pmatrix} \quad W^K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \quad W^V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

6.2 Step 1 — queries, keys and values

A 0/1 row of X simply selects and adds rows of a weight matrix. Enemy $= (1,0,1,0)$ selects rows 1 and 3: $q_{\text{enemy}} = (1,0) + (1,0) = (2,0)$. Tank $= (0,1,0,1)$ selects rows 2 and 4: $q = (0,1)$, $k = (1,0) + (1,0) = (2,0)$, $v = (0,1) + (0,1) = (0,2)$. Advances $= (1,0,0,1)$ selects rows 1 and 4: $q = (1,0)$, $k = (1,1)$, $v = (1,1)$.

$$Q = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad K = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}, \qquad V = \begin{pmatrix} 2 & 1 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}.$$

Note that $q_{\text{enemy}} = (2,0)$ points the same way as $k_{\text{tank}} = (2,0)$: enemy is about to find tank highly relevant.

6.3 Step 2 — scores and scaling

$$S = QK^{\mathsf{T}} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 4 & 2 \\ 2 & 0 & 1 \\ 0 & 2 & 1 \end{pmatrix}, \qquad \frac{S}{\sqrt{2}} = \begin{pmatrix} 0 & 2.8284 & 1.4142 \\ 1.4142 & 0 & 0.7071 \\ 0 & 1.4142 & 0.7071 \end{pmatrix}.$$

Row = query, column = key. For example enemy→tank $= q_{\text{enemy}} \cdot k_{\text{tank}} = 2 \cdot 2 + 0 \cdot 0 = 4$.

6.4 Step 3 — softmax

Row enemy: the exponentials of the scaled scores are $(1,\ 16.9188,\ 4.1133)$, with sum 22.0321, giving weights $(0.0454,\ 0.7679,\ 0.1867)$. Doing the same for every row:

$$A = \begin{pmatrix} 0.0454 & 0.7679 & 0.1867 \\ 0.5760 & 0.1400 & 0.2840 \\ 0.1400 & 0.5760 & 0.2840 \end{pmatrix}.$$

Every row sums to 1. Enemy attends mostly to tank (77%); tank looks back at enemy (58%); the verb "advances" looks to its subject tank (58%).

6.5 Step 4 — blend the values

Row advances: $z = 0.14(2,1) + 0.576(0,2) + 0.284(1,1) = (0.28 + 0 + 0.284,\ 0.14 + 1.152 + 0.284) = (0.5641,\ 1.5760)$. In full,

$$Z = AV = \begin{pmatrix} 0.2775 & 1.7679 \\ 1.4359 & 1.1400 \\ 0.5641 & 1.5760 \end{pmatrix} \begin{matrix} \leftarrow \text{enemy} \\ \leftarrow \text{tank} \\ \leftarrow \text{advances} \end{matrix}.$$

Each output is pulled towards the values of the words it attended to: z_{advances} now carries mostly tank information — the verb "knows" what is advancing.

6.6 Why divide by $\sqrt{d_k}$

Derivation — the size of a dot product

If the components q_i and k_i are independent with mean 0 and variance 1, each product $q_i k_i$ has mean 0 and variance $E[q_i^2]E[k_i^2] = 1$, so $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and variance d_k; its typical size is $\sqrt{d_k}$ (the paper's footnote 4). Simulation agrees: standard deviations 1.42, 8.04 and 22.59 for $d_k = 2$, 64 and 512, against $\sqrt{d_k} = 1.41$, 8.00 and 22.63. Dividing by $\sqrt{d_k}$ restores unit scale whatever the model size.

Without scaling, large scores push softmax into saturation: one weight near 1, the rest near 0, and the softmax gradient $\partial a_j / \partial s_m = a_j(\delta_{jm} - a_m)$ is then close to zero everywhere, so learning stalls. In our example the unscaled row enemy, $\text{softmax}(0, 4, 2) = (0.016, 0.867, 0.117)$, is already sharper than the scaled $(0.045, 0.768, 0.187)$ at $d_k = 2$; at $d_k = 512$ scores are about 16 times larger and the unscaled softmax would be almost one-hot.

6.7 Masked (causal) self-attention

A model that generates text left to right must not use words it has not yet written. Before the softmax a mask is added:

$$A = \text{softmax}\Big(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\Big), \qquad M_{ij} = \begin{cases} 0, & j \le i \\ -\infty, & j > i. \end{cases}$$

Since $e^{-\infty} = 0$, future positions receive zero weight. In our example enemy can see only itself (weight 1.00); tank re-normalises over enemy and tank, $(4.1133, 1)/5.1133 = (0.8044, 0.1956)$; advances is unchanged. The decoder's first sub-layer uses this mask, as does every GPT layer; encoders use none.

7 Multi-Head Attention

Definition — multi-head attention (Vaswani et al., Figure 2, right)

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O,$$
$$\text{head}_i = \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V).$$

Each head has its own projections and so asks its own question; the h heads run in parallel, their outputs are placed side by side, and W^O mixes them back to d_{model} numbers per word. The paper uses $h = 8$, $d_k = d_v = d_{\text{model}}/h = 64$, so the concatenation has $8 \times 64 = 512$ columns and W^O is 512×512; the total cost is about that of one full-width head. A staff cell is a fair analogy: intelligence, logistics and fires officers read the same report and each extracts different links.

7.1 Worked example: a second head

$$W_2^Q = W_2^K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad W_2^V = \begin{pmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \Rightarrow Q_2 = K_2 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}, \; V_2 = \begin{pmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{pmatrix}.$$

$$S_2 = Q_2 K_2^{\mathsf{T}} = \begin{pmatrix} 4 & 0 & 2 \\ 0 & 4 & 2 \\ 2 & 2 & 2 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 0.7679 & 0.0454 & 0.1867 \\ 0.0454 & 0.7679 & 0.1867 \\ 0.3333 & 0.3333 & 0.3333 \end{pmatrix}, \quad Z_2 = \begin{pmatrix} 0.2775 & 1.7225 \\ 1.7225 & 0.2775 \\ 1.0000 & 1.0000 \end{pmatrix}.$$

Row enemy: scores $(4, 0, 2)/\sqrt{2}$, exponentials $(16.92, 1, 4.11)$, weights $(0.768, 0.045, 0.187)$. Head 1 linked words to *each other* (enemy→tank, advances→tank). Head 2, with $W_2^Q = W_2^K$, matches each word mainly with *itself*, preserving its own meaning, while advances spreads evenly. The same input, different weights, different patterns.

7.2 Concatenate and project

$$\text{Concat}(Z_1, Z_2) = \begin{pmatrix} 0.2775 & 1.7679 & 0.2775 & 1.7225 \\ 1.4359 & 1.1400 & 1.7225 & 0.2775 \\ 0.5641 & 1.5760 & 1.0000 & 1.0000 \end{pmatrix}.$$

For hand arithmetic take $W^O = I$, so the multi-head output equals the concatenation; a trained W^O would mix the two heads. Shapes: each head 3×2; concatenation 3×4; after W^O, 3×4, the shape of X. The output must keep the width of X so that it can be *added* back to the input (the residual connection) and so that the next layer can be stacked on top. Choosing $d_v = d_{\text{model}}/h$ makes W^O square: the paper's $8 \times 64 = 512$ matches our $2 \times 2 = 4$.

7.3 Parameter count of one attention layer

Per head: W_i^Q, W_i^K, W_i^V hold $3 \times d_{\text{model}} \times d_k$ weights; over h heads that is $3d_{\text{model}}^2$ when $hd_k = d_{\text{model}}$; W^O adds d_{model}^2. With biases a layer has $4d_{\text{model}}^2 + 4d_{\text{model}}$ parameters: $4(512^2) + 4(512) = 1{,}050{,}624$ for the base model, independent of the number of heads.

7.4 What heads learn

In trained models some heads link verbs with subjects and objects ("crossed" finds "unit"); some attend to the previous word, a positional pattern made possible by the encoding; the paper's appendix shows heads that follow long-distance dependencies and resolve pronouns. Probing studies found heads tracking syntax and copying repeated phrases; many heads are redundant and can be pruned with little loss.

Pitfall — attention maps are not explanations

High attention weight does not prove that a word caused the output: values are mixed again by W^O, the FFN and later layers. Treat heat-maps as hypotheses about what a model uses, not as evidence of why it decided.

8 The Architecture of *Attention Is All You Need*

Figure 1 of the paper has two towers, read bottom to top. The left **encoder** reads the source sentence; the right **decoder** writes the target sentence; a Linear layer and softmax at the top give output probabilities. Every block outputs vectors of the same size, $d_{\text{model}} = 512$. The numbers **(1)**–**(6)** below follow the slides.

8.1 Following one sentence through the figure

The task in the paper is English→German translation: "enemy tank advances" → "Feindlicher Panzer rückt vor" (words simplified to one token each), base model, $n = 3$ source tokens and $m = 5$ decoder inputs.

	Block	What happens	Shape
(1)	input embedding	3 IDs look up 3 rows, scaled by $\sqrt{512} = 22.6$	3×512
(2)	+ positional encoding	add sinusoidal codes for positions 0, 1, 2	3×512
(3)	encoder layer $\times 6$	8-head self-attention (3×3 scores per head), Add & Norm, FFN $512 \to 2048 \to 512$, Add & Norm	3×512 (memory)
(4)	outputs (shifted right)	$\langle s \rangle$ Feindlicher Panzer rückt vor: embed, add position, masked self-attention (5×5, lower triangle)	5×512
(5)	encoder–decoder attention	queries from 5 decoder positions, keys and values from the 3-word memory (5×3)	5×512
(6)	Linear + softmax	a score for each of ≈37,000 tokens, turned into probabilities	$5 \times 37{,}000$

8.2 (1) (2) Embedding and positional encoding

Token IDs become learned 512-dimensional vectors. The paper **shares one weight matrix** between the input embedding, the output embedding and the final pre-softmax Linear layer, and multiplies embeddings by $\sqrt{d_{\text{model}}}$ so that the position codes, whose values lie in $[-1, 1]$, do not drown them. The positional encoding (the wave symbol and $\oplus$ in the figure) is added element by element on both towers; dropout with $P_{\text{drop}} = 0.1$ is applied to the sums.

8.3 (3) The encoder layer

Each of the $N = 6$ identical layers (with separate weights) has two sub-layers. **Multi-head self-attention:** the three arrows entering the block are Q, K and V, all copies of the same input; every source word attends to every source word with no mask. **Position-wise feed-forward network:**

$$\text{FFN}(x) = \max(0,\, xW_1 + b_1)\, W_2 + b_2, \qquad 512 \to 2048 \to 512,$$

the same two-layer ReLU network applied to each word separately — attention mixes words, the FFN processes each word. It holds $2(512)(2048) + 2048 + 512 = 2{,}099{,}712$ parameters, twice the attention layer. Each sub-layer is wrapped in **Add & Norm**:

$$\text{output} = \text{LayerNorm}\big(x + \text{Sublayer}(x)\big), \qquad \text{LayerNorm}(h) = \gamma \odot \frac{h - \mu}{\sigma} + \beta,$$

where μ and σ are the mean and standard deviation of the components of the single vector h. The bypass arrow is the residual connection: the original vector passes straight through and the sub-layer adds a correction; during training the gradient flows through the "+" unchanged, so 6 (or 96) layers still train.

Worked example — Add & Norm on the running example

Add the multi-head output to the input:

$$H = X + \mathrm{MultiHead}(X) = \begin{pmatrix} 1.2775 & 1.7679 & 1.2775 & 1.7225 \\ 1.4359 & 2.1400 & 1.7225 & 1.2775 \\ 1.5641 & 1.5760 & 1.0000 & 2.0000 \end{pmatrix}.$$

Row enemy: $(1,0,1,0) + (0.2775, 1.7679, 0.2775, 1.7225)$. Its mean is $\mu = 6.0454/4 = 1.5113$; deviations $(-0.2338, 0.2566, -0.2338, 0.2112)$; variance 0.0550; $\sigma = 0.2344$. Normalised: $(-0.998,\ 1.094,\ -0.998,\ 0.901)$. All rows ($\gamma = 1$, $\beta = 0$):

$$\mathrm{LN}(H) = \begin{pmatrix} -0.998 & 1.094 & -0.998 & 0.901 \\ -0.635 & 1.513 & 0.240 & -1.118 \\ 0.082 & 0.115 & -1.506 & 1.309 \end{pmatrix}.$$

Each row then passes through the FFN and a second Add & Norm, completing one encoder layer; the 3×4 result is ready for layer 2.

8.4 (4) Decoder input and masked self-attention

In training the correct German sentence is fed to the decoder **shifted right** behind a start symbol (*teacher forcing*):

position	1	2	3	4	5
decoder input	$\langle s \rangle$	Feindlicher	Panzer	rückt	vor
word to predict	Feindlicher	Panzer	rückt	vor	$\langle /s \rangle$

Position t must predict word t while seeing only words $1, \ldots, t-1$. The causal mask stops position 3 ("Panzer") from attending to "rückt" and "vor"; in the paper's words, masking plus the one-position offset ensure that "predictions for position i can depend only on the known outputs at positions less than i". All five predictions are then made in one parallel pass. At inference there is no target: the decoder starts from $\langle s \rangle$ and feeds back each word it produces.

8.5 (5) Encoder–decoder (cross) attention

The decoder's second sub-layer takes its **queries** from the decoder ("which source words do I need for my next German word?") and its **keys and values** from the encoder's final output — the two arrows entering from the left tower. It is the only connection between the towers. The scores are 5×3 (decoder positions $\times$ source words) and no mask is needed, because the whole source sentence is known. The weights act as a learned **word alignment**: both halves of the separable German verb "rückt … vor" look back to "advances".

8.6 (6) Linear and softmax

Each final decoder vector (512 numbers) is multiplied by a $512 \times V$ matrix (shared with the embedding table) to give one logit per vocabulary token, about 37,000 for the paper's English–German model, and softmax turns these into probabilities. Training minimises the cross-entropy $-\ln P(\text{correct next word})$ at every position, with **label smoothing** $\epsilon_{ls} = 0.1$: the target puts probability 0.9 on the correct word and spreads 0.1 over the others, which made the model less over-confident and improved translation quality. At inference the paper used **beam search** with

beam size 4, keeping the four most probable partial translations at each step rather than only the single best word.

8.7 Three uses of the same attention

	Encoder self-attention (3)	Decoder masked self-attention (4)	Encoder–decoder attention (5)
Queries from	source words	target words so far	target words so far
Keys, values from	source words	target words so far	encoder output
Mask	none	causal	none
Scores (example)	3×3	5×5, lower triangle	5×3
Question answered	how do the English words relate?	what have I written so far?	which English words do I translate next?

The formula is identical in all three places; only the sources of Q, K, V and the mask differ.

9 The Paper's Settings and Results

	Base	Big
layers N (each tower)	6	6
d_{model}	512	1024
d_{ff}	2048	4096
heads h	8	16
$d_k = d_v$	64	64
dropout	0.1	0.3
parameters	65 M	213 M
training on 8 P100 GPUs	12 hours	3.5 days
BLEU, WMT 2014 English–German	27.3	**28.4**
BLEU, WMT 2014 English–French	38.1	**41.8**

The big model beat the best previous English–German results, including ensembles, by more than 2 BLEU at a fraction of their training cost (the English–French big model used dropout 0.1). A rough count for the base model: each encoder layer has $1{,}050{,}624 + 2{,}099{,}712 + 2 \times 1{,}024 = 3{,}152{,}384$ parameters and each decoder layer, with two attention sub-layers and three layer norms, 4,204,032; six of each give 44.1 million, and the shared $37{,}000 \times 512$ embedding adds 18.9 million, about 63 million in all — close to the paper's 65 million, the difference depending on the exact vocabulary.

Training recipe. Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$; about 4.5 million English–German sentence pairs; and a learning rate that warms up and then decays:

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min\!\left(\text{step}^{-0.5},\ \text{step} \cdot \text{warmup}^{-1.5}\right), \qquad \text{warmup} = 4000.$$

With $d_{\text{model}}^{-0.5} = 0.0442$: at step 1,000 the rate is $0.0442 \times 1000 \times 4000^{-1.5} = 1.75 \times 10^{-4}$; it peaks at step 4,000 at $0.0442 \times 4000^{-0.5} = 6.99 \times 10^{-4}$; by step 16,000 it has fallen to 3.49×10^{-4}. The warm-up avoids large, destabilising updates while Adam's statistics are still unreliable.

10 Three Families of Transformer Models

	Encoder-only	Decoder-only	Encoder–decoder
Part of Figure 1	left tower	right tower without **(5)**	both towers
Examples	BERT, RoBERTa	GPT series, Claude, Llama	original Transformer, T5, BART
Attention	bidirectional	causal mask	bidirectional encoder; causal decoder with cross-attention
Trained to	fill in masked words	predict the next token	map input text to output text
Defence example	tag units, places and equipment in intercepts	draft a SITREP from field notes	translate captured documents

The building block is the same throughout; the families differ only in the mask, the wiring and the training objective. Scale grew quickly: BERT-base about 110 million parameters (2018), GPT-2 1.5 billion (2019), GPT-3 175 billion (2020).

11 Generation and Training

11.1 Next-token generation

A decoder-only model runs the prompt through all blocks; the last position's vector times an output matrix gives a logit for every vocabulary token; softmax gives probabilities; a token is chosen and appended; and the loop repeats. **Temperature** τ divides the logits before the softmax.

Worked example — temperature

After "The convoy crossed the" suppose the logits for bridge, river, road, enemy, tank are $(3.2, 2.1, 1.8, 0.4, -0.5)$.
$\tau = 1$: $(0.601, 0.200, 0.148, 0.037, 0.015)$. $\tau = 0.5$: $(0.850, 0.094, 0.052, 0.003, 0.001)$ — more predictable. $\tau = 2$: $(0.404, 0.233, 0.200, 0.100, 0.063)$ — more varied.

11.2 Training

For every position t of every training sentence the loss is the cross-entropy of the true next token,

$$L = -\frac{1}{T}\sum_{t=1}^{T} \ln P(w_{t+1} \mid w_1, \ldots, w_t).$$

If the true word above is "bridge", that position contributes $-\ln 0.601 = 0.51$. The causal mask lets one forward pass produce a prediction, and a loss term, at *every* position simultaneously; this parallelism is why transformers train so efficiently. The gradient then flows back through the softmax, W^O, every head's W^Q, W^K, W^V, the FFNs and the embedding table by the same chain rule as Lecture 6. After pre-training a model may be **fine-tuned** on a smaller task-specific dataset; **instruction tuning** and **training from human feedback** teach it to follow requests helpfully and safely; or it may be used without further training by placing examples in the prompt (**in-context learning**).

12 Defence and Wargaming Applications

Defence & wargaming application — where transformers are used

Text and intelligence: summarising reports, extracting units, locations and equipment from intercepts, translating captured documents, searching archives by meaning.
Imagery (Vision Transformer): a 224×224 SAR or EO image cut into 16×16 patches gives $14 \times 14 = 196$ tokens, each with a positional encoding; attention links distant parts of a scene, such as vehicles and the road network they use. Lecture 7 compares CNNs and transformers.
Sensor fusion and tracks: each radar track or report becomes a token; attention learns which tracks relate (formation flying, escort–target pairs).
Wargaming and decision support: policies over sets of units of any size, analysis of game logs to find decisive moments, natural-language interfaces to simulations. Lectures 10–12 use neural networks as the function approximators of reinforcement learning.

Pitfall — limits to brief commanders on

Hallucination: fluent, confident, wrong statements.
Prompt injection: hostile text inside a document can redirect the model.
Distribution shift: trained on open text, deployed on jargon and new theatres.
Cost: attention is $O(n^2)$ in sequence length; doubling the context quadruples the score matrices.

13 Common Misconceptions

Pitfall — seven errors to avoid

"Attention is understanding." It is a weighted average learned to reduce prediction error.
"Q, K and V are three different inputs." In self-attention they are three projections of the same X.
"Positional encoding is concatenated." It is added element by element.
"Embedding dimensions have names." Only in toy examples.
"The model reads left to right." Only decoders are causal; encoders see the whole sequence.
"Softmax picks the best word." It gives a distribution; the decoding strategy (greedy, beam search, sampling) picks.
"Longer context is free." The score matrix is $n \times n$.

14 Summary of Equations

Input	$h_0 = \sqrt{d_{\text{model}}}\, E_{\text{tokens}} + PE$
Positional encoding	$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$
Projections	$Q = XW^Q$, $K = XW^K$, $V = XW^V$
Attention	$\text{softmax}(QK^\mathsf{T}/\sqrt{d_k} + M)\, V$
Multi-head	$\text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$
Sub-layer wrapper	$\text{LayerNorm}(x + \text{Sublayer}(x))$
Feed-forward	$\max(0, xW_1 + b_1)W_2 + b_2$
Attention parameters	$4d_{\text{model}}^2 + 4d_{\text{model}}$ per layer
Learning rate	$d_{\text{model}}^{-0.5} \min(\text{step}^{-0.5},\ \text{step} \cdot 4000^{-1.5})$
Loss	$-\ln P(\text{correct next token})$, averaged over positions

15 Check Your Understanding

Check your understanding — 1

With the toy table, compute lion − man + woman, where lion = $(0.85, 0, 0.90, 0.30)$. Which word is nearest?

Answer. $(0.85, 0, 0.90, 0.30) + (-1.80, 0, 0, 0) = (-0.95,\ 0,\ 0.90,\ 0.30)$. Cosine similarities: lioness 0.996, tigress 0.973, female 0.767. The nearest word is *lioness*.

Check your understanding — 2

Compute the positional encoding p_3 for $d_{\text{model}} = 4$.

Answer. $(\sin 3, \cos 3, \sin 0.03, \cos 0.03) = (0.1411,\ -0.9900,\ 0.0300,\ 0.9996)$.

Check your understanding — 3

In head 1 of the worked example, which word does "tank" attend to most, and with what weight?

Answer. "Enemy", with weight 0.576 (scaled scores 1.4142, 0 and 0.7071; exponentials 4.113, 1 and 2.028; sum 7.141).

Check your understanding — 4

Why must the multi-head output have the same width as X?

Answer. So that it can be added to X in the residual connection and so that identical layers can be stacked. W^O maps the concatenation of the heads back to d_{model}.

Check your understanding — 5

In Figure 1, where do the keys and values of block **(5)** come from, and why is no mask needed there?

Answer. From the encoder's final output (the memory). The whole source sentence is known before decoding starts, so nothing about it is "future"; the causal restriction applies only to the

target words, and is enforced in block **(4)**.

Check your understanding — 6

How many attention scores per head does a 1,000-token input need, and how does that compare with a 4,000-token input?

Answer. $1{,}000^2 = 10^6$ scores; for 4,000 tokens 1.6×10^7, sixteen times as many for four times the length.